Practical Guide to Everyday Cloud Operations Commands

Cloud Operations Engineers keep applications fast, reliable, and reachable around the clock. Master these core diagnostic commands to quickly troubleshoot infrastructure issues like a pro.

What you'll learn:

1. Checking Server Health and Performance
2. Inspecting Application Logs in Real Time
3. Monitoring Storage and Free Space
4. Testing Network and Database Connectivity

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Checking Server Health and Performance

As a Cloud Operations Engineer, keeping servers healthy is job number one. When a website gets slow, you need to check if CPU or RAM is maxed out. Tools like htop show an interactive, color-coded view of system usage in real time. It helps you quickly spot rogue processes consuming too many resources.

```
$htop
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Inspecting Application Logs in Real Time

When something breaks in production, application logs tell you the exact story. Instead of opening huge log files in an editor, stream live log output directly in your terminal. Filter by app name or pod name to isolate the error message right as it occurs. This lets you debug incidents in seconds instead of hours.

```
$kubectl logs -f deployment/backend-api --tail=50
```

3

TIP 3 OF 4

Monitoring Storage and Free Space

Disks filling up silently is one of the most common causes of middle-of-the-night server outages. Using human-readable disk monitoring commands helps you see total capacity, used space, and available gigabytes across all mounted drives. Running this regularly prevents surprise crashes caused by runaway log files.

```
$ df -h
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Testing Network and Database Connectivity

Applications frequently fail because security groups, firewalls, or DNS settings block network access. Netcat (nc) lets you test if a specific port is reachable on a remote server before blame gets assigned to application code. It sends a simple TCP handshake to confirm the network pathway is open.

```
$ nc -zv database.internal.net 5432
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always check disk space (df -h) first when services mysteriously stop working.
- + Stream live logs during deployments to spot crash loops immediately.
- + Test network connectivity before assuming application code is buggy.
- + Use human-readable flags like -h and -m for easier command line reading.
- + Keep diagnostic terminal commands saved in a personal command cheat sheet.

Watch Out For

- ! Blindly restarting servers without reading error logs first.
- ! Forgetting that log files grow over time, leading to 100% disk usage.
- ! Assuming network port errors are application bugs instead of firewall rules.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh